

Meeting Minutes

9/25/2024

Scribe: Brandon

Attendance: Austin, Brandon, Kabir, Connor, Boyd, Julie

Main Points: Understand how Matlab script works and how to get data from binary into usable format by our application

Walking through BD_inWater.m script:

- hardcoded filepath at top
- Hardcoded BD number (device that records)
- Couple of user inputs,
 - Expected binary file name
- 78/79 open binary file and read into 10x120,000 int 32 matrix, then transposes it
 - This is how we can test to see if our data conversion in python matches the matlab
- gdata() function takes in matrix and bd number and outputs
 - Basically outputs the configuration for the bluedrop with each conversion factor
 - Factors are shown on one spreadsheet Julie will send
 - Ppm var is in psi, we want it in kPa to keep constant units
 - other s are in 'g's,
 - Xy params are horizontal acceleration sensors(she will leave note on which horizontals are what), g's are vertical acceleration sensors
- alldatafig() plots all of the data
 - Contains "findpeaks()" which will find where the spikes are
 - Might see many peaks due to this which would make looking at the data weird. We will see this in the provided dataset (collected just off the shore of new castle)
 - Plots a scatterplot of the peaks
 - Asks which drop you want to analyze out of the peaks, have to pick number off of plot
 - Calls ptpick()
 - Truncates data before and after spike to zoom in on the actual drop, chops off junk
 - Has an if elif else to ensure we're not chopping off too much from the beginning or end
- Start and end of drop are used to zoom in on similar points for other variables
 - Picks out maximum point to decide which sensor to utilize
 - Different sensors are selected based off of what the "maximum" is, some sensors are more powerful than others so if one max's out we need to use another one
 - Only uses one accelerometer based off of maximum measured acceleration
 - Offset just used to help with integration (shifting to 0)
- Ignore p1 = ppm(start1:ent1)
- Penetrometer "comes to rest" when its back to 1g, so we use 2g (near 126) to detect this
 - findent2() does this^
- Then plots 2g and previously determined accelerometer based on selected spike

- Prompts user for “spike step”, data starts to increase at a certain point, essentially picking that point (x value in shown plot) where it begins to increase (obvious with sand)
 - User should be able to input this, but **in theory** could be automated, someone did before but didn’t work
 - Would be helpful to prompt back to user to show what they selected to ensure they have the right spot
- Zooms in on spike, converts to g’s, gives us a time vector based on points per second.
- cumtrapz() is a cumulative trapezoidal integral, python lib should have similar functionality (**can be utilized through SciPy**)
- Now integrate deceleration with time,
 - Find the maximum and flip it around to integrate the time it takes to “come to rest”
 - Vel should be closed to 0 (close to rest)
- Then calculate depth in respect to velocity integrating again
- areafind() finds area based on certain factors from drop, tiptype, atype, d, tlength
 - Can be basically 1-1 in python
- Divide force by surface area
- Don’t worry about buoyancy!
- $\text{Mass} * \text{accel} * \text{grav} = F_{be}$ (force) in air
- Uses ldivide to do force/area which results in pressure (bearing capacity of soil)
- For the next part, she will provide us the equations
 - “The best way to get out of soft stuff is to floor it”, so the faster you go the stronger the soil is
 - Need to convert this from a dynamic problem to a static problem, she will provide this all to us
- User input or select menu to input srfK, strain rate correction
- Followed by more plotting
- 213/214 gets weird, could get automated
 - Need to input start/end timesteps
 - The tip is a small surface area, so sometimes we get infinity errors here.
 - We need to pick the point where the “decrease” ends and “increases”
 - Basically just selecting a window, code can just suggest end points to user
- Ignore all pp2/pp3 things, next part is basically just restating the variables and corrected velocities utilizing the start/end variables.
- Converts dynamic bearing capacity from pascals to kilopascals
- Ignore next qsbcc section, as long as we can select calibration factors and plot we should be okay for now
 - A lot of reordering data to the right format
- **If we can set it up to plot everything over the drop, then the “final fig” can be ignored**
 - Depth on y axis, velocity on x
 - Depth vs acceleration
 - Depth vs dyn bearing cap
 - On this plot we can put the corrected and dynamic bearing capacity
 - This is from the start of the spike to the end of the spike (g=1; rest)

- Summary data needs to be changed, its no longer really relevant
- Save at end contains all of the processed data, saving the first set of processed data would be great. This is the “original conversion format”. Using a CSV would work great

For “data conversion” story, want to ensure that units are correct

9/23/2024:

Scribe: Austin

Attendance: Austin, Brandon, Kabir, Connor, Boyd

Main Points: Sprint Planning

Questions to answer:

Why is this sprint valuable?

- Need to understand the current code, and initialize our UI with selecting data and converting it to a common form.

What can be done for this sprint?

- Understanding of current code, selecting file, and converting file

How will the chosen work get done?

- Kabir and Austin have been assigned the 2 main story points, and will dish out the individual tasks amongst the team
- The entire team will grasp the current status of the code and understand the different file formats and existing processes

What is our “Definition of Done” (DoD)?

- Understand the current code, create initial UI with 1 button “select”, and have “select” button load some .bin /.pnl test data and convert to common format (likely .csv). This will be shown in our demo

Action Items:

- Look over “sponsor plan” sent after previous meeting with team
- Update the README.md file in the repository
- Define “done” for this sprint, estimate story points, assign stories
- Individually, flesh out stories and what tasks need to be done to complete them

What was done:

- Checked out sponsor document with her estimations
- Set up sprints in GitLab
- Estimate Weights
 - Pointed “Open Data”, “Data Conversion” with individual subtasks in gitlab
- Defined tasks

9/20/2024:

Scribe: Kabir

Attendance: Austin, Brandon, Boyd, Connor, Kabir

Main Points: Create charter and finalize stories

Action Items:

- Charter:

- Condensed ideas and information from class meeting on Monday and sponsor meeting on Wednesday.
- Created MOV, finalized MVP
 - A lot of discussion about MOV vs Success Criteria, how specific each should be and how they interact

- Stories:

- Finalized user stories that were started on Monday
 - Created a new potential user, someone who would like to analyze a dataset that has been previously analyzed and uploaded by someone else
 - Created more stories and flushed them out
 - Created expanded story
 - Made WBS
-

9/18/2024:

Scribe: Austin and Brandon

Attendance: Ausin, Boyd, Brandon, Connor, Kabir, Julie, Emma

Main Points: Understand scope and vision from sponsor, clarify questions identified in first team meeting, outlined meeting schedule for rest of the semester

Future meetings on Wednesdays around same time, ideally in the same space

Julie will be traveling the first week in November (the 6th)

Aim for Python first or Matlab if it is easier

- Ease of use is the main thing

Existing scripts are mainly in Matlab and Excel

- 5 individual code (one of them has the most)

How they want to use

- 1 stop shop
- Select analysis process
- Select data to analyze

How do we get the data?

- Data comes in files over ethernet
 - One file for each minute

- Select the time stamps of the data to analyze

Visualization

- Raw data
- Integrate data

2 different paths

- Generating formatted data from raw data
- Going from data in a format to visuals

Binary Data

- 120k x 10 32 bit integer matrix
- Columns 1 and 2 don't mean much
- The rest of the columns have accelerometers or pressure sensors
- Spit out file and time stamp of drops

Eliminate

- Specify the penetrometer in use and have the calibrations in the application so they can be set automatically
 - Or be able to enter in constants

Need to be able to define configuration per penetrometer. constants are known for existing penetrometers.

Would be ideal to upload whatever calibration factors you want into the app. Plug in bin data, pick calibration factors, pick out data around spikes to isolate drop spots.

After processing data, could display each drop spot for the user to select. Isolate each drop spot's data into their own files. Easy way to select file for drop.

Be able to take user input for parameters to generating graphs

Matlab files are about 9000KB, binary files are about

Potential for comparing previous data to new data, but not a priority.

Simplifying more detailed excel programs is more of a second semester thing

Data comes in as binary. As penetrometer falls and hits soil, it reads data files and opens visual.

Drop samples occur at a 2000 pts per second. Need to save each spike individually.

Figuring out where the spikes are in the data to save a lot of time.

1 file per minute.

Windows specifically for desktop use.

Existing dropView software should be put in UI, should be fine for now.

Want to view each integration/intermediary steps of deceleration, velocity, and bearing output of soil

Want to be able to fiddle with data factors

Probability/uncertainty: second semester type thing. Add probabilities of different types of soil.

Goal: To have a total understanding of what is required for the MVP by the end of the Semester

Questions to ask:

- What do the existing python and matlab scripts do? How is excel used?
- What are the most valuable portions of this project to you?
- How should the user go about utilizing the product? Walk us through what you expect when you open the program

Action Items:

- Ask questions above
- Set up weekly or biweekly recurring meeting and potentially backup times

Highly researched piece of equipment that many people do research on. Code is chaos to use.

Doesn't know what would be best. Most written in MATLAB and excel, but ideally python should be used.

Processing is a series of steps in matlab as a series of functions.

5 individual files, some unwritten, some missing and some correlations. Emma has been doing this in excel. Already ranked.

Open data, select type of processing, spit out the kind of soil.

—

9/16/2024:

Scribe: Brandon

Attendance: Ausin, Boyd, Brandon, Connor, Kabir

Main Points: Identify questions to ask sponsor, set up GitLab board, define outline of charter

Project description:

Julie Paprocki. User Interface for Free Fall Penetrometers

Free fall penetrometers are an emerging tool used to perform early site investigation for offshore wind and the marine renewable energy sector. These penetrometers employ accelerometers that record at high frequency (2 kHz) to estimate parameters of interest, such as soil characterization, undrained shear strengths, friction angles, and consolidation characteristics. However, these processing steps exist in disparate codes. The advancement of this technology would greatly benefit from a unified user interface to extract and process the raw data. In this project, I would provide the team with the base codes for initial data processing and calculation of the relevant parameters as well as the frameworks that have been developed. The intended product is a user interface that implements these codes into a single system that can be shared across multiple users so the processing and characterization process is streamlined.

Skills: reading existing matlab and/or python coding. Ability to implement some probability into the existing codes so that there is uncertainty incorporated into the existing analysis.

What is the problem?

There exists multiple different types of data processing and calculation to estimate parameters of interest from free fall penetrometers. These systems are not uniform and cannot be shared across multiple users which makes more complex analysis difficult.

What is the vision; what value does your solution provide?

Is to provide a uniform user interface that combines the existing processing and calculation systems into a single desktop application that can be shared and expanded upon by multiple people over time.

Simplify and unify the data analyzing process for data collected from penetrometers.

MVP

Creating a desktop application that combines the existing systems.

Epics

- User interface
- Unifying the systems

Logistics

- Communication will be done preferably in person but we also have a discord server
- Meeting minutes will be done in shared Google Doc (this document)
- Artifacts will be in markdown which will be a part of the GitLab repository
 - Can be converted to PDF for submissions
 -

Question for Julie

- Existing systems? What languages, how many scripts, how complicated?
- How do we expect the user to interface with the problem?
 - Terminal application?
 - **GUI?**
- Who is/are the main user/users?
- What does “shared by multiple users” entail? Same system, or data uploaded to a shared area online? **Import/Export**
- How could/will the existing systems be expanded upon in the future? **Python**
- What does implementing probability to account for uncertainty entail within the current systems?
Stretch goal for identifying soil

Action Items

- Create and setup Gitlab Repository
- Draft Epics
- Draft user personas/stories
- Enter stories as issues into GitLab
 - Rank stories based on importance (Discuss with Julie)
 - More personas??
- Set up meeting time with Julie

User Personas

Persona: OE Researcher(s)

Description: Collects data and processes the data via the user interface needs the system to be easy to learn because they may not be using it for a long time.

Persona: Administrator (Research Lead, Julie)

Description: Handles the system and updates it with new modules, needs the system to be updateable/upgradeable because the cast of researchers is changing.

User Stories

**** means high priority*

*** means medium priority*

** means low priority*

Nothing means not part of MVP

***As an OE Researcher, I want to be able to share my data with other OE Researchers so that collaboration can be done more efficiently.*

****As an OE Researcher I want to be able to select different data processing methods within the same application so that I do not need to use multiple systems to process the same data.*

****As an OE Researcher, I want a simple unified interface so that I can focus on analytics rather than data pipelines.*

****As an OE Researcher, I want to be able to generate a visual of the data so that I can properly analyze my results.*

***As an OE Researcher, I want to be able to account for uncertainty within my data so that I can be more confident in my analysis.*

**As an Administrator, I want to be able to easily add more data processing modules so that this tool can be updated in the future.*

***As an Administrator, I want to be able to easily edit data processing modules so that this tool can be adapted in the future.*

As an Administrator, I want to be able to view a history of data input/manipulation so that I can see how the system has evolved over time.

***As an OE Researcher, I want to be able to be shown isolated drop points, so I don't need to search through all of the data.*

**As an OE Researcher, I want to be able to import previously processed data, so I can view someone else's results.*

***As an OE Researcher, I want to be able to input save constants for my data, so that I don't need to enter them every time I use the program.